BÖLÜM 9 / MODULE 9

İleri Seviye MAANG Tasarım Problemleri

Advanced MAANG Distributed System Designs

Uber Yolculuk Eşleştirme (H3 Altıgenleri), Dağıtık Metrik Sistemi (TSDB), DynamoDB Klonu, Dağıtık Kuyruk (Zero-Copy), Stripe Ödeme Sistemi ve Canlı Skor Tablosu (SkipList).

Uber Ride-Hailing (H3 Hexagons), Time-Series Monitoring (Prometheus TSDB), Distributed Key-Value Store (DynamoDB), Kafka Clone, Stripe Payments, and Gaming Leaderboard (SkipLists).

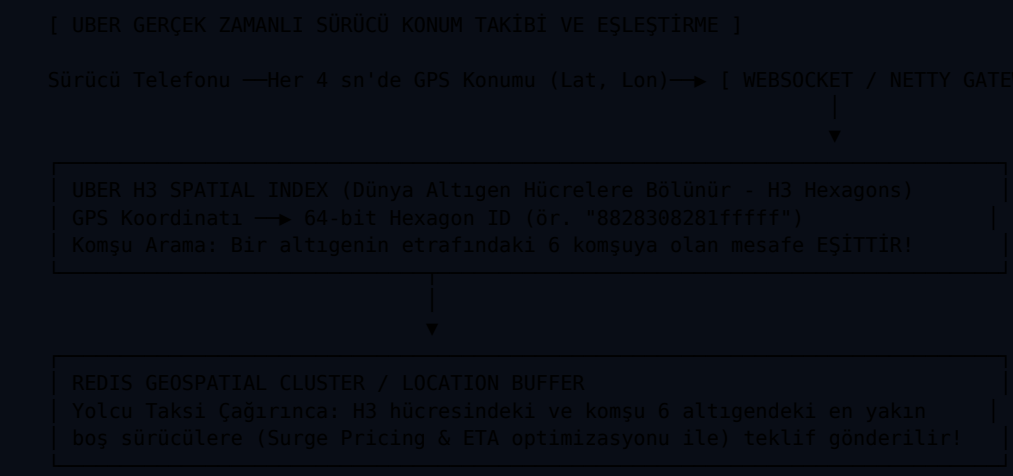
49

BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Gerçek Zamanlı Yolculuk Eşleştirme (Uber / Lyft): Uber H3 Altıgenleri ve Konum Takibi

Designing Ride-Hailing (Uber / Lyft): Uber H3 Hexagons & Real-Time Dispatch

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kare bir ızgarada merkezdeki bir kutunun yan komşusu ile çapraz komşusunun mesafesi aynı değildir (çapraz komşu %41 daha uzaktır!). Bu yüzden Uber dünyayı karelere değil, arıların peteği gibi altıgenlere (H3 Hexagonal Grid) bölmüştür! Bir altıgenin etrafındaki 6 komşusunun da merkeze olan uzaklığı BİREBİR AYNIDIR. 1 milyon taksi şoförü her 4 saniyede bir 'Ben şu altıgendeyim' diye sinyal gönderir. Siz 'Taksi çağır' butonuna bastığınızda, sistem sizin bulunduğunuz altıgene ve etrafındaki 6 komşu peteğe bakar; 1 saniyede en yakın 3 taksiciyi ekranınıza düşürür!

🇬🇧 ENGLISH (ELI5)

On a square grid, the distance to an edge neighbor differs from a diagonal neighbor (diagonals are 41% further away!). To eliminate this mathematical distortion, Uber divided the entire planet into a honeycomb of hexagons (Uber H3 Spatial Index). Every neighbor of a hexagon shares the EXACT same distance to its center! 1 million drivers broadcast their GPS location every 4 seconds. When a rider taps 'Request Ride', the dispatch engine evaluates drivers in the rider's active hexagon and its 6 adjacent neighbor cells, matching the optimal driver in under 1 second!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Uber Mimarisi: 1) Konum Güncelleme Hacmi (Location Ingestion): 1M aktif sürücü x her 4 saniyede 1 GPS pingi = 250,000 QPS yazma yükü! Bu yükü doğrudan ilişkisel veritabanına yazmak sistemi eritir. Sürücü konumları hafif WebSocket veya gRPC bağlantılarıyla alınır ve bellek içi Redis Geospatial (`GEOADD`) veya RAM tabanlı H3 spatial index'te tutulur. 2) Neden Uber H3 (Altıgen Mimarisi)?: H3, dünyayı hiyerarşik 16 çözünürlük seviyesinde altıgenlere bölen açık kaynaklı bir kütüphanedir. Altıgenlerin tüm komşularına olan mesafeleri sabittir; bu sayede dinamik fiyatlandırma (Surge Pricing) ve sürücü arama algoritmaları çok daha homojendir. 3) Eşleştirme Motoru (Dispatch Engine): Yolcu istek attığında, H3 k-ring (örneğin 1 km yarıçapındaki altıgenler) taranır. Aday sürücüler arasından ETA (Tahmini varış süresi), sürücü puanı ve rota uygunluğuna göre en iyi eşleşme yapılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Uber Scale Spatial Architecture: 1) Ingestion Scale: 1M active drivers sending GPS pings every 4 seconds generates 250,000 writes/sec! Relational engines collapse under this load. Ingestion routes over persistent WebSockets/gRPC into distributed in-memory Redis clusters or custom H3 memory rings. 2) Why Uber H3 (Hexagonal Topology)?: Unlike square grids where diagonals are √2 times longer, hexagons exhibit uniform distance to all 6 adjacent neighbors. H3 provides 16 discrete hierarchical resolutions (Resolution 8 spans 450m, perfect for urban dispatch). Surge pricing boundaries map cleanly to H3 cells. 3) Real-Time Matchmaker: When a rider requests a pickup, the dispatcher executes an `H3.kRing(originHex, radius=2)` query. Candidate drivers are ranked via real-time routing engines (taking one-way streets, traffic congestion, and driver acceptance rates into account) before dispatching the offer.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Simüle Edilmiş Uber H3 Altıgen Eşleştirme Modeli
import java.util.*;

public class UberDispatchEngine {
    // Sürücü Modeli
    public record Driver(String driverId, String h3HexId, boolean isAvailable) {}

    private final Map<String, List<Driver>> hexagonalGrid = new HashMap<>();

    public void updateDriverLocation(Driver driver) {
        hexagonalGrid.computeIfAbsent(driver.h3HexId(), k -> new ArrayList<>()).add(driver);
    }

    // Yolcunun altıgen hücresindeki ve komşu hücrelerdeki müsait sürücüleri bul
    public List<Driver> findNearbyDrivers(String riderHexId, List<String> neighborHexIds) {
        List<Driver> candidates = new ArrayList<>();

        // 1. Yolcunun kendi hücresine bak
        for (Driver d : hexagonalGrid.getOrDefault(riderHexId, List.of())) {
            if (d.isAvailable()) candidates.add(d);
        }
        // 2. Komşu 6 altıgene bak (H3 k-Ring)
        for (String neighbor : neighborHexIds) {
            for (Driver d : hexagonalGrid.getOrDefault(neighbor, List.of())) {
                if (d.isAvailable()) candidates.add(d);
            }
        }
        return candidates;
    }
}

void main() {
    var dispatch = new UberDispatchEngine();
    dispatch.updateDriverLocation(new UberDispatchEngine.Driver("drv_1", "hex_kadikoy_c1", true));
    dispatch.updateDriverLocation(new UberDispatchEngine.Driver("drv_2", "hex_kadikoy_c2", true)); //
Komşu altıgen

    var nearby = dispatch.findNearbyDrivers("hex_kadikoy_c1", List.of("hex_kadikoy_c2",
"hex_kadikoy_c3"));
    IO.println("Eşleşen Müsait Sürücü Sayısı: " + nearby.size());
    for (var d : nearby) IO.println("• Sürücü #" + d.driverId() + " (Hücre: " + d.h3HexId() + ")");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta sürücülerin saniyelik konumlarını PostgreSQL veya MySQL'e yazacağınızı ASLA söylemeyin! Saniyede 250.000 disk yazması veritabanını felç eder. Sürücünün anlık konumu uçucu (ephemeral) veridir; sadece RAM'de (Redis) tutulur. Sadece yolculuk bittiğinde tamamlanan rota veritabanına arşivlenir.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch writing raw GPS pings directly to a persistent relational database! 250,000 writes/sec will incinerate database IOPS. Current driver location is ephemeral data stored exclusively in in-memory spatial caches (Redis); only finalized trip routes commit to persistent databases.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Uber H3 altıgen ızgarası komşular arası mesafeyi eşit tutarak arama algoritmasını optimize eder.

• 250.000 QPS'lik konum akışı ilişkisel DB yerine tamamen bellek içi Redis'te tutulur.

• Yolculuk eşleştirmede H3 k-ring hücreleri taranarak en düşük ETA'ya sahip sürücü seçilir.

🎯 Key Takeaways for the Interview

• Uber's H3 hexagonal grid ensures uniform neighbor distances, eliminating diagonal distortions.

• High-frequency GPS telemetry streams into in-memory Redis clusters, bypassing disk bottlenecks.

• Dispatchers search surrounding H3 k-ring clusters to identify drivers minimizing rider ETA.

50

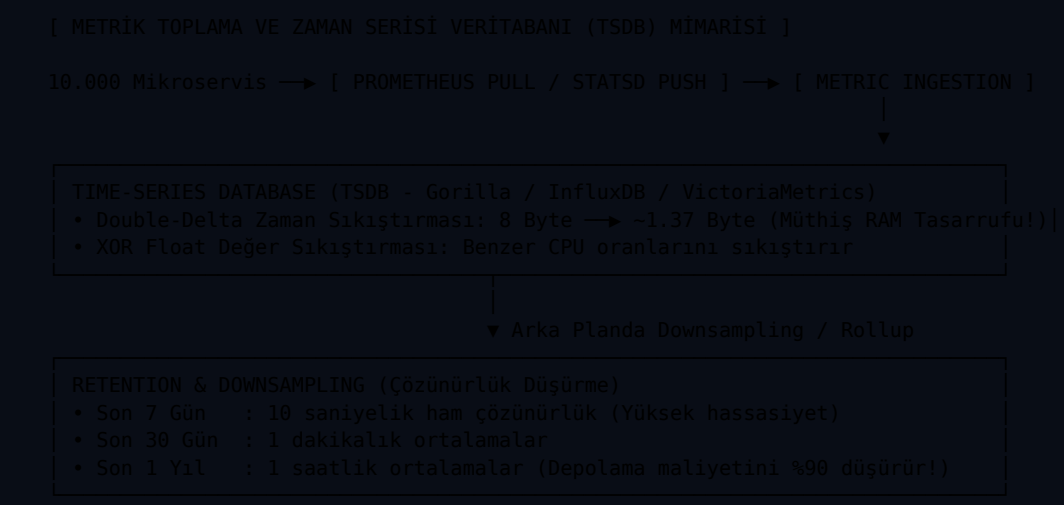
BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Dağıtık Metrik ve İzleme Sistemi (Datadog / Prometheus): TSDB ve Downsampling

Designing Metrics & Monitoring (Datadog / Prometheus): TSDB & Aggregations

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir hastanın yoğun bakımda saniyede 1 kez nabzını ölçtüğünüzü düşünün. Bir günde 86.400 nabız verisi çıkar. 1 ay sonra doktor hastanın 3 hafta önceki durumuna bakmak istediğinde saniye saniye 2.5 milyon nabız verisini tek tek okumak istemez; '3 hafta önce her günkü ortalama nabız neydi?' diye sorar! Dağıtık metrik sistemleri de böyledir: Sunucuların CPU kullanımını saniye saniye kaydeder (TSDB); ancak 1 ay geçince o saniyelik verileri silip yerine '1 saatlik ortalama' kaydını bırakır (Downsampling). Böylece hard diskler milyonlarca gereksiz sayıyla dolup taşmaz!

🇬🇧 ENGLISH (ELI5)

Imagine logging an intensive care patient's heart rate every single second. That generates 86,400 numbers a day. Six months later, the doctor reviewing recovery progress doesn't need to scroll through 15 million individual second-by-second data points; they only care about the daily average and peak trends! That is what a Time-Series Monitoring System (Prometheus, Datadog) does: it ingests high-resolution telemetry every 10 seconds, but as data ages past 30 days, background jobs roll up seconds into hourly averages (Downsampling), slashing storage footprint by 90%!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Metrik Sistemi Mimarisi: 1) Pull vs Push Modeli: Prometheus 'Pull' modelini kullanır (Sunucuları kendisi tarar / scrape eder; istemciye yük binmez). Datadog/CloudWatch 'Push' modelini kullanır (Agent metrikleri merkezi API'ye fırlar). 2) Zaman Serisi Veritabanı (TSDB) Sıkıştırması (Facebook Gorilla Mimarisi): Zaman serisi verisi `(Timestamp, Value)` çiftidir. Zaman damgaları 'Double-Delta Compression' ile (ör. her 10 saniyede bir gelen veri için farkın farkı sıfır olur ve tek bir 0 bitiyle temsil edilir); Float değerler ise 'XOR Floating-Point Compression' ile sıkıştırılır. 16 baytlık bir metrik noktası ortalama 1.37 bayta düşürülür (10 kat bellek sıkıştırması!). 3) Downsampling & Rollup: Veri eskidikçe çözünürlük kademeli düşürülür (Raw 10s -> 5m Rollup -> 1h Rollup).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architecture of Scale Metrics Systems: 1) Pull vs Push: Prometheus implements a Pull architecture (periodically scrapes `/metrics` endpoints via service discovery; auto-detects dead hosts). Datadog/StatsD uses Push (agents dispatch UDP/TCP packets to gateways; ideal for ephemeral serverless lambdas). 2) TSDB Compression (Facebook's Gorilla Paper): Time-series data comprises `(timestamp, float_value)` streams. Gorilla compresses timestamps via Double-Delta encoding (D = (t[i] - t[i-1]) - (t[i-1] - t[i-2])); periodic intervals evaluate to zero, encoded in a single bit. Float values are compressed by XORing against previous values. Slashes 16-byte tuples down to 1.37 bytes per metric point in memory. 3) Downsampling & Rollups: Background compaction jobs aggregate granular metrics into downsampled buckets (10s raw for 7 days -> 1m rollup for 30 days -> 1h rollup for 1 year), bounding long-term storage growth.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Basitleştirilmiş Zaman Serisi Metrik Veri Noktası ve Rollup Modeli
import java.util.*;

public class MetricsRollupEngine {
    public record MetricPoint(long timestampSeconds, double value) {}

    // Saniyelik verileri 1 dakikalık ortalama noktasına indirgeme (Downsampling)
    public static MetricPoint rollupToMinute(List<MetricPoint> rawPoints) {
        if (rawPoints.isEmpty()) return null;
        double sum = 0;
        for (var p : rawPoints) sum += p.value();
        double avg = sum / rawPoints.size();
        return new MetricPoint(rawPoints.getFirst().timestampSeconds(), avg);
    }
}

void main() {
    // 1 dakikalık sürede toplanan 3 saniyelik CPU verisi: %40, %60, %50
    var raw = List.of(
        new MetricsRollupEngine.MetricPoint(1700000000L, 40.0),
        new MetricsRollupEngine.MetricPoint(1700000010L, 60.0),
        new MetricsRollupEngine.MetricPoint(1700000020L, 50.0)
    );

    var rolledUp = MetricsRollupEngine.rollupToMinute(raw);
    IO.println("Ham Veri Noktası Sayısı: " + raw.size());
    IO.println("Downsampled 1-Dk Ortalama: %" + rolledUp.value() + " [Zaman: " +
rolledUp.timestampSeconds() + "]");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta metrikleri standart ilişkisel veritabanına (MySQL/PostgreSQL) kaydedeceğinizi söylemeyin! Milyonlarca metrik satırı B-Tree indekslerini patlatır ve disk doldurur. Metrikler için daima TSDB (Prometheus, TimescaleDB, VictoriaMetrics, ClickHouse) önerilmelidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch storing high-velocity time-series metrics inside a traditional relational database (MySQL/PostgreSQL). The volume of writes will shred B-Tree indexes and consume petabytes of disk. Prescribe purpose-built TSDBs like VictoriaMetrics or ClickHouse.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- TSDB veritabanları zaman serisi verisini Gorilla double-delta algoritmasıyla 10 kat sıkıştırır.
- Prometheus Pull modeli sunucu keşfiyle hafif çalışırken, Push modeli sunucusuz ortamlar için uygundur.
- Downsampling politikaları eskiyen verileri özetleyerek disk maliyetlerini %90 düşürür.

🎯 Key Takeaways for the Interview

- TSDBs compress timestamp-value streams by 10x using Gorilla double-delta and XOR algorithms.
- Pull architecture suits stateful microservice discovery; Push fits ephemeral serverless functions.
- Downsampling policies aggregate aging historical metrics, bounding storage costs sustainably.

51

BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Dağıtık Web Önbelleği (Distributed Key-Value Store / DynamoDB Klonu)

Designing a Distributed Key-Value Store (DynamoDB Clone)

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram

```
[ AMAZON DYNAMO DÖNÜM NOKTASI MİMARİSİ (THE DYNAMO PAPER) ]

1. Veri Bölümleme : Tutarlı Karışıklık Halkası (Consistent Hash Ring + Vnodes)
2. Yüksek Yazma   : Sloppy Quorum & Hinted Handoff (Düğüm ölse bile komşuya yaz!)
3. Çakışma Çözümü : Vektör Saatleri (Vector Clocks) & İstemci Çözümlemesi
4. Arka Plan Onarım: Merkle Trees & Anti-Entropy (Arka planda sessizce eşitleme)
5. Düğüm Keşfi    : Gossip Protocol (Dedikodu Protokolü ile merkeziyetsiz sağlık kontrolü)
```

💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Amazon Dynamo, dünyanın en sağlam kargo ağı gibidir: Amazon'un kuralı şudur: 'Müşteri sepetine ürün eklerken sistem ASLA hata vermeyecek!' Bir düğüm arıza yapsa bile paket hemen yanındaki komşu düğüme emanet edilir (Sloppy Quorum & Hinted Handoff). Arızalı düğüm kendine geldiğinde emanetçi düğüm 'Bu paket senindi, buyur al' der. İki şube arasında anlaşmazlık çıkarsa Vektör Saatleri devreye girer. Sunucular aralarında fısıldaşarak (Gossip Protocol) kimin ayakta olduğunu birbirine haber verir. Tamamen merkeziyetsiz, lideri olmayan (Leaderless) ölümsüz bir mimaridir!

🇬🇧 ENGLISH (ELI5)

Amazon Dynamo is an un-killable decentralized courier grid designed around Amazon's prime directive: 'A customer adding an item to their shopping cart must NEVER see an error screen!' If the designated server node is down, the adjacent neighbor temporarily accepts the write (Sloppy Quorum & Hinted Handoff). Once the dead node recovers, the neighbor hands the package back. If conflicting updates clash, Vector Clocks preserve causal history. Nodes continuously whisper among themselves like neighbors at a fence (Gossip Protocol) to detect cluster topology and outages without a centralized master!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Werner Vogels ve ekibinin ünlü Amazon Dynamo Paper (2007) tasarımı MAANG mülakatlarının zirvesidir: 1) Sloppy Quorum & Hinted Handoff: Yazma sırasında belirlenen N düğümden biri erişilemezse, halkadaki bir sonraki sağlıklı düğüm veriyi geçici olarak kabul eder (`hinted handoff`). Düğüm ayağa kalkınca veri asıl sahibine teslim edilir (Erişilebilirlik tavan yapar). 2) Anti-Entropy with Merkle Trees: Düğümler arasındaki veri tutarsızlıklarını tüm veriyi ağdan taşımadan bulmak için Merkle Ağaçları (Hash Trees) kullanılır; ağaçların kök hash'leri uyuşuyorsa veriler eşittir. 3) Dedikodu Protokolü (Gossip Protocol): Merkezi bir ZooKeeper/etcd olmadan düğümler rastgele seçtikleri komşularla durum bilgilerini paylaşarak küme üyeliğini merkeziyetsiz olarak ($O(log N)$ sürede) senkronize eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Amazon Dynamo Paper (SOSP 2007) is the magnum opus of distributed systems architecture: 1) Sloppy Quorum & Hinted Handoff: During node partitions, if healthy nodes within the primary preference list fall below quorum W, the coordinator writes to fallback nodes outside the list with a 'hint' metadata tag. Once the primary recovers, the fallback pushes the mutation home, preserving write availability above all. 2) Anti-Entropy via Merkle Trees: Replicas synchronize divergent key ranges by comparing cryptographic Merkle Tree hash roots; branches are traversed only where hashes diverge, slashing network bandwidth. 3) Gossip Protocol: Nodes execute periodic peer-to-peer heartbeat exchanges with randomly selected nodes, propagating membership and failure detector states across the cluster in $O(log N)$ time without a central master coordinator.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Merkle Tree (Özet Ağacı) Mantığı ile Hızlı Veri Uyuşmazlığı Tespiti
import java.security.MessageDigest;

public class MerkleTreeSim {
    public static String hash(String input) {
        try {
            var md = MessageDigest.getInstance("SHA-256");
            return HexFormat.of().formatHex(md.digest(input.getBytes())).substring(0, 8);
        } catch (Exception e) { return ""; }
    }

    public static String computeRootHash(String dataBlockA, String dataBlockB) {
        String hashA = hash(dataBlockA);
        String hashB = hash(dataBlockB);
        // İki yaprağın hash'ini birleştirip üst düğümü oluştur:
        return hash(hashA + hashB);
    }
}

void main() {
    // İki farklı sunucudaki Merkle Kök Hash'lerini karşılaştır:
    String server1Root = MerkleTreeSim.computeRootHash("User:Ali", "Order:101");
    String server2Root = MerkleTreeSim.computeRootHash("User:Ali", "Order:101"); // Veriler aynı
    String server3Root = MerkleTreeSim.computeRootHash("User:Ali", "Order:999"); // Veri farklı!

    IO.println("Sunucu 1 ve 2 Eşit mi? " + server1Root.equals(server2Root) + " (Kök: " + server1Root +
")");
    IO.println("Sunucu 1 ve 3 Eşit mi? " + server1Root.equals(server3Root) + " (Uyuşmazlık Var!
Senkronize et)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta DynamoDB'nin 'Liderli (Master-based)' bir mimari olduğunu iddia etmeyin! Dynamo lidersizdir (Leaderless); her düğüm eşit yetkilere sahiptir. Bu sayede tek hata noktası (SPOF) barındırmaz.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never describe Dynamo as a master-slave system. Dynamo is strictly decentralized and leaderless; all nodes act as peers, eliminating single points of failure.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Sloppy Quorum ve Hinted Handoff düğüm arızalarında bile yazma erişilebilirliğini korur.
- Merkle Trees, düğümler arasındaki veri farklarını minimum ağ trafiğiyle senkronize eder.
- Gossip Protocol merkezi bir koordinatör olmadan küme sağlığını merkeziyetsiz yönetir.

🎯 Key Takeaways for the Interview

- Sloppy Quorum and Hinted Handoff guarantee write availability during node downtime.
- Merkle Trees identify replica divergence with minimal network bandwidth overhead.
- The Gossip protocol maintains decentralized cluster membership and failure detection.

52

BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Dağıtık Mesaj Kuyruğu (Distributed Message Queue / Kafka Klonu) Tasarımı

Designing a Distributed Message Queue (Kafka Clone): Zero-Copy & Commit Log

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ KAFKA KLONU PERFORMANS SIRRI: ZERO-COPY DISK-TO-NETWORK ]

GELENEKSEL YÖNTEM (4 KULLANICI ALANI - KERNEL KOPYASI, YAVAŞ!):
Disk ──1. DMA──► OS Cache ──2. CPU──► JVM Heap ──3. CPU──► Socket Buffer ──4. DMA──► NIC

KAFKA ZERO-COPY YÖNTEMİ (LINUX sendfile(): SIFIR JVM KOPYASI!):
Disk ──1. DMA Kopya──► OS PageCache ──2. DMA Kopya──► Network Interface Card (NIC)
(CPU ve JVM tamamen atlanır; diskten doğrudan ağ kartına ışık hızında akar!)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Normalde bir bilgisayarın diskindeki bir dosyayı internete göndermek, bir kargocunun paketi depodan alıp kendi ofis masasına getirmesi, paketi açması, tekrar bantlaması ve sonra kamyona taşıması gibidir (4 kez gereksiz yere taşınır). Kafka'nın Sıfır Kopyalama (Zero-Copy) sihri ise kargocuyu aradan çıkarıp depodan çıkan raylı sistemi doğrudan kamyonun kasasına bağlamaktır! Veri asla işletim sisteminin veya Java'nın hafızasına kopyalanıp vakit kaybetmez; doğrudan diskten ağ kablosuna fırlar!

🇬🇧 ENGLISH (ELI5)

Traditional file transmission over the network is like a warehouse worker loading a box onto their office desk, unboxing it, inspecting it, taping it shut again, and then carrying it to the delivery truck (4 redundant data copies and context switches!). Kafka's Zero-Copy optimization connects a direct gravity conveyor chute from the warehouse shelf straight into the back of the cargo truck! Data never touches JVM user-space memory; the Linux kernel routes raw disk PageCache bytes directly onto the Network Interface Card (NIC) at hardware line speed!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Dağıtık Mesaj Kuyruğu Tasarımının 3 Performans Direği: 1) Sıralı Disk Erişimi (Sequential IO): Diskler rastgele okumada (Random Seek) yavaştır ancak sıralı yazmada (Sequential Append) SSD'lerde 2-3 GB/sn hızlara ulaşır (RAM hızına çok yakındır). Kafka tüm mesajları sadece kütüğün sonuna ekler (Append-only). 2) Linux sendfile() (Zero-Copy): `transferTo()` sistem çağrısıyla veriler OS PageCache'ten doğrudan NIC buffer'ına kopyalanır. Context Switch (Kullanıcı alanı - Çekirdek geçişi) sayısı 4'ten 2'ye, veri kopyası sayısı 4'ten 2'ye düşer; CPU tüketimi sıfırlanır. 3) Toplu İşleme (Batching & Compression): Mesajlar tek tek değil, 64 KB'lık paketler halinde LZ4/ZSTD ile sıkıştırılarak ağdan aktarılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The 3 engineering pillars of Kafka's throughput mastery: 1) Sequential Disk IO: Mechanical and NVMe storage drives deliver massive bandwidth (gigabytes/sec) when written sequentially, rivaling raw DRAM speeds. Kafka leverages append-only commit logs exclusively. 2) Linux Zero-Copy (`sendfile` system call): Traditional reads transfer bytes: Disk -> PageCache -> User Space -> Socket Buffer -> NIC (4 buffer copies, 4 context switches). The `sendfile()` primitive transfers bytes directly from PageCache to the Network Interface Card via DMA, executing zero user-space memory copies and zero CPU load. 3) Batch Compression: Producers aggregate micro-messages into 64KB batches, compressing with LZ4/Snappy before transmission, maximizing network MTU utilization.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Zero-Copy (FileChannel transferTo) Simülasyonu
import java.nio.channels.*;

public class ZeroCopyDemonstrator {
    // Java NIO FileChannel.transferTo() arka planda Linux sendfile() kullanır!
    public static void zeroCopyTransfer(FileChannel sourceFile, WritableByteChannel targetSocket)
throws Exception {
        long position = 0;
        long count = sourceFile.size();

        // Veri JVM User-Space belleğine HİÇ girmeden doğrudan diskten ağ soketine uçar!
        while (position < count) {
            long transferred = sourceFile.transferTo(position, count - position, targetSocket);
            position += transferred;
        }
        IO.println("✔ Zero-Copy Transferi Tamamlandı! Boyut: " + count + " bytes");
    }
}

void main() {
    IO.println("Zero-Copy Metodu: FileChannel.transferTo() Hazır");
    IO.println("Avantaj: Sıfır JVM Garbage Collection, Sıfır CPU Buffer Kopyası!");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Mesajları hızlı olsun diye RAM'de (JVM Heap) saklarım' demeyin! JVM Heap'te milyonlarca mesaj saklamak Garbage Collection (GC) duraklamalarına yol açar ve sunucuyu kilitler. Kafka gibi sistemler işletim sisteminin PageCache'ini kullanarak disk ve RAM'i birlikte en verimli şekilde kullanır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose holding high-volume message queues within the JVM heap. Millions of heap objects trigger catastrophic Stop-The-World garbage collection pauses. Real systems delegate caching entirely to the underlying OS PageCache.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Sıralı disk yazması (Sequential IO) rastgele bellek erişiminden bile daha hızlı olabilir.
- Linux sendfile() Zero-Copy tekniği CPU ve JVM hafıza kopyalama yükünü tamamen sıfırlar.
- Toplu mesajlaşma (Batching) ağ başlık yükünü minimize ederek throughput'u katlar.

🎯 Key Takeaways for the Interview

- Sequential disk writes achieve multi-gigabyte throughput, outpacing random memory access.
- Linux sendfile() zero-copy streams bytes directly from disk cache to the network card.
- Batching micro-messages into compressed frames maximizes network packet density.

53

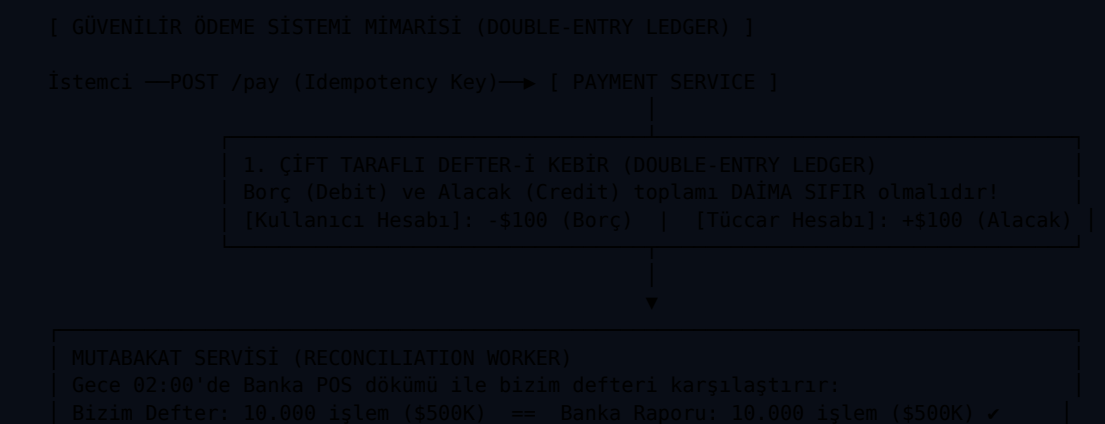
BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Ödeme Sistemi (Payment Gateway / Stripe): Çift Taraflı Muhasebe ve Mutabakat

Designing a Payment System (Stripe): Double-Entry Bookkeeping & Reconciliation

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Cüzdanınızdan 100 TL çıkarıp bir market sahibine verdiğinizde ne olur? Para havadan yok olmaz veya gökten zembille inmez! Sizin cebinizden 100 TL eksilir (-100 TL), marketçinin kasasına 100 TL girer (+100 TL). İkisini toplarsanız sonuç SIFIRDIR! 500 yıldır Venedikli tüccarların kullandığı 'Çift Taraflı Muhasebe' (Double-Entry Bookkeeping) budur. Stripe gibi devasa bir ödeme sisteminde tek bir 'bakiye' sayısı tutulmaz; tüm para hareketleri borç ve alacak olarak kaydedilir. Gece olunca da bankanın hesap dökümü ile bizim kayıtlarımız kuruşu kuruşuna karşılaştırılır (Mutabakat - Reconciliation)!

🇬🇧 ENGLISH (ELI5)

When you hand a $100 bill to a grocery clerk, cash doesn't materialize out of thin air or vanish into the void! Your wallet decreases by -$100 (Debit), while the register increases by +$100 (Credit). The net sum of the universe remains exactly ZERO! This is 500-year-old Double-Entry Bookkeeping. In a world-class payment platform like Stripe, you never store a simple mutable `balance = 500` column. You record immutable debit and credit ledger entries. At night, an automated auditor (Reconciliation Engine) cross-references internal ledgers against bank settlement files cent-for-cent!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Ödeme Sistemi Mimarisi Gereksinimleri: 1) Idempotency Garantisi: Ağ kopmalarında mükerrer para çekilmesini önlemek için istemci tarafından gönderilen `Idempotency-Key` zorunludur. 2) Çift Taraflı Muhasebe (Double-Entry Bookkeeping): Asla `UPDATE accounts SET balance = balance - 100` yapılmaz! Her işlem en az iki bacaklı bir defter kaydıdır (Ledger Entry): Biri Borç (Debit), diğeri Alacak (Credit). Sistem genelindeki tüm borç ve alacakların toplamı her an tam olarak sıfır (0) olmak zorundadır. 3) Mutabakat (Reconciliation): Bankalar ve PSP'ler (Stripe, Adyen, Visa) her gece CSV/XML formatında hesap mutabakat dosyaları (Settlement Files) gönderir. Arka plandaki mutabakat motoru, bizim yerel defterimizdeki işlemlerle bankanın bildirdiği işlemleri kuruşu kuruşuna eşleştirir; eksik veya hatalı işlemlerde alarm çalar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Payment Gateway Architecture: 1) Guaranteed Idempotency: Mitigates double-charging during network retries via client-supplied `Idempotency-Key` headers locked inside distributed Redis stores. 2) Double-Entry Bookkeeping Ledger: Strictly avoids mutable balance columns (`balance = balance - 100`). Money movements are immutably logged across dual ledger legs: Every transaction creates matching Debit and Credit entries across distinct accounts. Invariant: ∑ Debits - ∑ Credits = 0 across the global ledger at all times. 3) Reconciliation Engine: Financial partners (Visa, Chase, Adyen) transmit end-of-day settlement files. An automated asynchronous reconciliation pipeline cross-audits internal ledger transactions against partner clearing logs, flagging uncaptured funds or discrepancies for manual treasury review.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Çift Taraflı Muhasebe Defteri (Double-Entry Bookkeeping) Modeli
import java.math.BigDecimal;
import java.util.List;

public class DoubleEntryLedger {
    public enum EntryType { DEBIT, CREDIT }
    public record LedgerEntry(String accountId, EntryType type, BigDecimal amount) {}

    public static boolean validateTransaction(List<LedgerEntry> entries) {
        BigDecimal balance = BigDecimal.ZERO;
        for (LedgerEntry entry : entries) {
            if (entry.type() == EntryType.DEBIT) {
                balance = balance.subtract(entry.amount());
            } else {
                balance = balance.add(entry.amount());
            }
        }
        // Borç ve alacak toplamı SIFIR olmalıdır!
        return balance.compareTo(BigDecimal.ZERO) == 0;
    }
}

void main() {
    var amount = new java.math.BigDecimal("150.00");
    var entries = List.of(
        new DoubleEntryLedger.LedgerEntry("ACC_USER_ALİ", DoubleEntryLedger.EntryType.DEBIT, amount),
        new DoubleEntryLedger.LedgerEntry("ACC_MERCHANT_XYZ", DoubleEntryLedger.EntryType.CREDIT,
amount)
    );

    boolean balanced = DoubleEntryLedger.validateTransaction(entries);
    IO.println("Muhasebe Kaydı Dengeli mi? " + balanced + " (Borç = Alacak)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta para işlemlerini saklarken `FLOAT` veya `DOUBLE` veri tipi kullanacağınızı ASLA söylemeyin! Kayan noktalı sayılar (Floating-point) ikili sistemde yuvarlama hataları yaratır (0.1 + 0.2 = 0.30000000000000004). Milyonlarca dolarlık sistemlerde açık çıkar! Para daima `BIGINT` (kuruş/cent cinsinden) veya Java `BigDecimal` ile tutulmalıdır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never use FLOAT or DOUBLE primitives for currency in software engineering! Binary floating-point arithmetic introduces rounding inaccuracies that leak money. Always store currency as 64-bit BIGINT integers representing cents or Java `BigDecimal` with explicit rounding modes.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Idempotency Key ödeme sistemlerinde mükerrer para çekimini engelleyen ilk savunma hattıdır.
- Çift Taraflı Muhasebe (Double-Entry) paranın asla yoktan var veya vardan yok olmamasını garanti eder.
- Mutabakat (Reconciliation) motoru iç defter ile banka dökümlerini karşılaştırarak tutarlılık sağlar.

🎯 Key Takeaways for the Interview

- Idempotency keys prevent double-charging users during client network retries.
- Double-entry bookkeeping ensures every debit is mirrored by an exact credit.
- Reconciliation pipelines audit internal transactional state against external bank settlements.

54

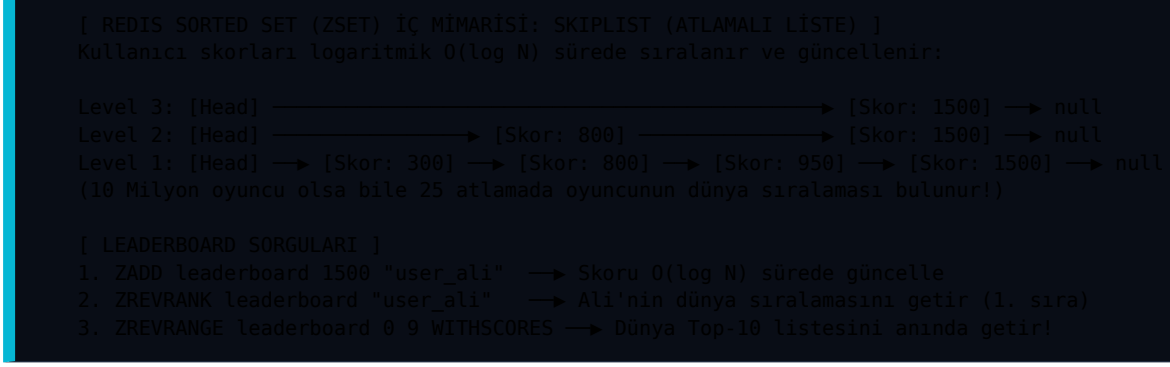
BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Canlı Oyun Lider Tablosu (Gaming Leaderboard): Redis Sorted Sets ve SkipList

Designing a Real-Time Gaming Leaderboard: Redis ZSET & SkipLists

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

10 milyon kişinin aynı anda oynadığı bir mobil oyunu düşünün. Herkes canavar öldürdükçe puanı artıyor. 'Ben dünyada kaçıncı sıradayım?' dediğinizde, 10 milyon kişiyi sıfırdan küçükten büyüğe sıralamaya kalkarsanız oyun donar! Redis Sorted Set işte bu yüzden sihirli bir ekspres trene (SkipList) benzer: En alt katta her istasyon vardır; ama üst katlarda sadece ana istasyonlar vardır. Tren üst kattan son hızla atlayarak gider ve 10 milyon kişi arasından sizin sıranızı göz açıp kapayıncaya kadar (1 milisaniyede) bulup çıkarır!

🇬🇧 ENGLISH (ELI5)

In a mobile game with 10 million active players, scores fluctuate every microsecond. If querying 'What is my global rank?' forced the database to run an $O(N log N)$ sort across 10 million rows, servers would melt! Redis Sorted Sets solve this using a multi-level express subway called a SkipList. The bottom track stops at every single station, while elevated tracks skip 10 or 100 stations at a time. The algorithm rides the express tracks, finding any player's exact global rank among 10 million contenders in under 1 millisecond ($O(log N)$)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Oyun Lider Tablosu Mimarisi: 1) Neden İlişkisel DB Yetmez? Milyonlarca oyuncunun olduğu bir tabloda `SELECT COUNT(*) WHERE score > my_score` sorgusu tam bir felakettir; her puan değişiminde indeks kilitlenir. 2) Redis ZSET (Sorted Set): İki veri yapısının mükemmel birleşimidir: A) HashMap: Oyuncu adına göre skoru $O(1)$ sürede bulur. B) SkipList (Atlamalı Liste): Skorları sıralı tutar. Skor güncelleme (`ZADD`), sıralama bulma (`ZREVRANK`) ve aralık okuma (`ZREVRANGE`) $O(log N)$ sürede biter. 10 milyonluk bir tabloda $log_2(10^7) approx 24$ işlemde sonuç alınır. 3) Eşit Puanların Ayrılması: İki oyuncu aynı puana sahipse (ör. ikisi de 1500), skora zaman damgası eklenerek erken puan alan öne geçirilir: `EfektifSkor = Skor - (Timestamp / 10^13)`.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Scale Gaming Leaderboard Architecture: 1) Relational DB Breakdown: Executing `SELECT COUNT(*) FROM leaderboard WHERE score > user_score` executes full table/index scans that crush concurrency. 2) Redis ZSET (Sorted Set Duality): Blends a Hash Map (for $O(1)$ player lookup) with a probabilistic multi-level SkipList (for $O(log N)$ insertions, removals, and rank lookups). For a leaderboard of 10 million gamers, traversing the SkipList requires at most $approx 24$ pointer hops. 3) Common Leaderboard APIs: `ZADD leaderboard score user_id` updates a score; `ZREVRANK leaderboard user_id` returns global rank; `ZREVRANGE leaderboard 0 9` fetches the global Top-10. 4) Tie-Breaking Mechanics: When two players share identical scores (e.g. 1500 points), prioritize the player who achieved it first by subtracting a scaled epoch timestamp: `CompositeScore = Score + (1 - (Timestamp / 10^13))`.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Eşit Puanları Zaman Damgasıyla Çözen Lider Tablosu Skoru
public class LeaderboardScoreCalculator {
    // Eşit puanlarda önce ulaşanı üst sıraya taşıyan formül:
    public static double computeCompositeScore(long points, long timestampMillis) {
        // Puan tamsayı kısmında kalır, zaman damgası kesirli kısma küçük bir eksi olarak eklenir
        double timePenalty = (double) timestampMillis / 1_000_000_000_000.0;
        return points + (1.0 - (timePenalty % 1.0));
    }
}

void main() {
    long score = 2500;
    long timeAli = 1700000000000L;       // Ali saat 12:00'de ulaştı
    long timeVeli = 1700000060000L;      // Veli 1 dakika sonra ulaştı

    double aliComposite = LeaderboardScoreCalculator.computeCompositeScore(score, timeAli);
    double veliComposite = LeaderboardScoreCalculator.computeCompositeScore(score, timeVeli);

    IO.println("Ali'nin Skoru:  " + String.format("%.8f", aliComposite));
    IO.println("Veli'nin Skoru: " + String.format("%.8f", veliComposite));
    IO.println("Ali Veli'den Önde mi? " + (aliComposite > veliComposite)); // TRUE!
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Dünya Top-100 listesini her oyuncunun ekranında saniyede 1 kez polling ile yenilerim' demeyin! 5 milyon aktif oyuncu saniyede 5 milyon istek demektir. Top-100 listesi CDN veya yerel bellek önbelleğinde 5 saniyelik bir TTL ile tutulmalı, kullanıcının kendi sırası WebSocket ile sadece değiştikçe iletilmelidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Polling the global Top-100 leaderboard every second from millions of mobile clients will saturate Redis network bandwidth. Cache the Top-100 at CDN edge PoPs with a 5-second TTL, pushing personalized rank mutations over WebSockets only when milestones cross.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Redis ZSET (SkipList) 10 milyonluk kümelerde bile sıralamayı O(log N) sürede bulur.
- Eşit puanlarda (Tie-Breaking) zaman damgası eklenerek erken ulaşan ödüllendirilir.
- Top-100 gibi global listeler CDN seviyesinde kısa TTL ile önbelleğe alınarak Redis korunur.

🎯 Key Takeaways for the Interview

- Redis ZSETs leverage SkipLists to resolve user ranks in O(log N) time across millions of users.
- Composite scoring incorporates timestamp fractions to break score ties deterministically.
- Edge-caching Top-100 leaderboards protects Redis from millions of redundant read queries.